

내 컴퓨터를 꺼도 서비스는 살아있다

심리테스트 공유 서비스로 배우는 GitHub · Vercel · Supabase

몸

앱 코드 · 내 폴더

기억

GitHub · 코드의 역사

무대

Vercel · 24시간 실행

창고

Supabase · 모두의 데이터

맥박

push → 자동배포

그 앱, 지금도 살아있나요?

7차 할 일 앱 — URL은 살아있다. 그런데 여러분이 적은 할 일은 어디에?

살아있는 것 ✓

- 앱 화면과 URL — 무대(Vercel) 위에 있으니까
- 폰 홈 화면의 아이콘

감혀 있는 것 ✗

- 내가 적은 할 일 — **localStorage**, 그 브라우저 안
- 폰 바꾸면 사라짐 · 친구는 절대 못 봄
- "전체 사용자 통계" 같은 건 원리적으로 불가능

데이터가 '내 기기 안'에 사는 한 — 그건 **서비스**가 아니라 **앱**이다.

오늘 만드는 것 — 심리테스트 공유 서비스

강사 완성본 라이브 시연. 잘 보세요, 마지막 장면이 오늘의 전부입니다.

① 폰에서 테스트 풀기

질문 5개, 30초면 끝

② 결과 + 통계

"당신 유형은 전체 참여자의 23%"

③ 공유 링크 복사 → 수강생 한 명에게 카톡

그 자리에서 전송

④ 그 수강생 폰에서 — 강사의 결과가 열린다 ★

본인은 아무것도 안 풀었는데

친구 폰에서 내 결과가 보인다 — 이게 되려면 뭐가 필요할까? 그게 오늘의 이론입니다.

코드·실행·데이터에게 집을 준다

몸(앱) · 기억(GitHub) · 무대(Vercel) · 창고(Supabase) · 맥박(자동배포) — 오늘의 다섯 단어.

01 · 이론

세 집의 원리

클라이언트·서버·클라우드부터, 세 도구가 왜·무엇을·어떻게 하는지.

02 · 실습

세 집에 입주

내가 기획한 테스트를 바이브 코딩 → 기억·무대·창고에 입주.

03 · 결과물

내 서비스 URL

친구에게 보낼 공유 링크 + 창고에 쌓이는 진짜 응답들.

오늘부터 여러분이 만드는 건 앱이 아니라 — 내 컴퓨터를 꺼도 살아있는 서비스다.

- 이론 ① · 앱과 서비스는 다르다

인터넷은 손님과 주방이다

인터넷의 모든 일은 딱 두 역할 — 요청하는 쪽(클라이언트)과 응답하는 쪽(서버).



손님

클라이언트

브라우저·폰

"이 페이지 주세요" (요청)

요청 →
← 응답



주방

서버

24시간 켜진 컴퓨터

화면·데이터를 내놓다

지금까지 우리는(4차 Artifacts · 7차 localhost) — 내가 손님이자 주방이었다. 내 기기 안 자급자족. 그래서 남이 못 온다.

localhost의 정체 — "내 컴퓨터 안"

옆 사람 폰에서 안 열리는 게 당연 — 그 주소 자체가 '각자 자기 컴퓨터'를 가리키는 예약어니까.

필요 ①

24시간 켜진 컴퓨터

내 노트북은 덮으면 끝. 손님이 언제 올지 모른다.

필요 ②

전 세계가 찾는 주소

도메인 — "여기로 오세요"라고 부를 수 있는 이름.

필요 ③

안전한 연결

HTTPS 자물쇠 — 7차에서 "설치는 HTTPS에서만"이었던 이유.

셋 다 내 노트북엔 없다.
그래서 — **빌린다.**

클라우드 = 남의 컴퓨터를 빌려 쓰기

데이터센터에 컴퓨터를 수만 대 켜놓고 빌려주는 회사들. 단 — 빌려주는 게 서로 다르다.

우리말	회사	빌려주는 것	비유
기억	GitHub	코드 보관용 컴퓨터 (원본·역사)	일기장이자 금고
무대	Vercel	앱 실행용 컴퓨터 + 주소 + HTTPS	24시간 극장
창고	Supabase	데이터 보관용 컴퓨터 + 관리 화면	은행 금고

셋 다 무료로 시작하고, 셋 다 **GitHub 계정 하나로 로그인**된다 — GitHub가 개발 세계의 신분증인 이유.

왜 셋으로 나누나

코드는 '역사'가, 실행은 '안 꺼짐'이, 데이터는 '안 잃음 + 규칙'이 중요하다 — 성격이 다르면 집도 다르다.

8차 회수

한 집 = 한 책임

PILOT의 L(나눈다)이 서비스 규모로 커진 것.

자유

갈아끼우기

무대를 옮겨도 기억과 창고는 그대로. 분리의 힘.

실물

우리 채점기도 이렇게 산다

코드 GitHub · 무대 Railway · 창고 Supabase — 오늘 배우는 게 진짜 서비스의 표준 구조.

이 수업 리더보드가 실시간으로 뜨던 것 — **오늘 배우는 창고(Supabase)**가 한 일입니다.

최종_진짜_최종.js의 지옥에서 탈출

코드가 내 폴더에만 있으면 생기는 세 가지 일.

GitHub 없으면 X

- 노트북 고장 = 서비스 전체 중발
- "어제까진 됐는데"로 못 돌아감
- Vercel이 코드를 가져갈 주소가 없음 → 자동배포 불가

GitHub 있으면 ✓

- 원본 보관소 + 역사책 — 모든 버전이 남는다
- 아무 시점으로 복귀 가능
- 다른 서비스들과의 접선 장소 — 배포 파이프라인의 심장

6차 파일 변경기로 정리했던 그 지옥이 코드에서 재현되는 걸 — Git이 원천 차단한다.

버전 관리 = 게임 세이브

"지금 상태"를 이름 붙여 저장해두고, 언제든지 그 시점으로 돌아가는 기술.

Git 게임기

내 컴퓨터에서 도는 **세이브 프로그램**. 전 세계 개발자의 표준.

GitHub

클라우드 세이브

세이브 파일들을 올려두는 **클라우드 서버**. 올려야 '내 컴퓨터 밖'에 존재.

커밋

세이브 1회

스냅샷 + "뭘 바꿨는지" 메모. **좋은 메모 = 미래의 나에게 보내는 쪽지**.

push

올리기

내 세이브들을 GitHub에 올리기. 오늘 배울 **맥박의 방아쇠**.

브랜치

딴 길 세이브

복제해서 딴 길로 가보는 것 — 오늘은 본선(main) 하나만. '이런 게 있다'까지만.

GitHub 기능 지도

오늘 쓰는 건 위 4줄. 아래는 '이런 것도 있다'.

기능	뭐 하는 것	비유
repo	프로젝트 하나가 사는 방 — <code>github.com/아이디/프로젝트명</code>	집 주소
커밋·push	세이브 만들기 · 올리기	세이브 → 금고
히스토리	모든 커밋이 시간순으로 — 빨강(지운 줄)/초록(넣은 줄) 비교	타임머신
README	repo 첫 화면의 소개문 (4차 "사람한테 한 장")	집 문패
공개/비공개	public이면 누구나 열람 — 그대로 포트폴리오	오픈하우스
Issues·PR	협업용 게시판·검토 절차 — 오늘은 안 씀	회사 결재 라인

오늘 쓰는 동작은 딱 3개

명령어를 외우지 않는다 — Claude Code에게 말로 시키고, 무슨 일이 일어났는지만 이해한다.

1

"GitHub에 로그인해줘"

gh auth login — 처음 한 번,
브라우저 인증만 내가

2

"이 폴더를 새 저장소로 올려줘"

처음 한 번 —
repo 생성 + 첫 push

3

"커밋하고 push해줘"

이후 무한 반복 —
세이브 → 금고

확인 = 웹에서: repo 페이지 → **commits** → 커밋 클릭 → **빨강/초록** 비교. "뭘 고쳤는지"의 증거.

7차 배포는 택배, 오늘은 컨베이어 벨트

내 컴퓨터를 서버로 쓸 수는 없다 — 24시간·주소·HTTPS 전부 없으니까. 그래서 극장을 빌린다.

7차 방식 — CLI 택배 ✗

- `vercel --prod` — 완성본을 한 번 던지기
- 고칠 때마다 다시 보내야 한다
- 사람이 손으로 = 잊고, 늦고, 실수한다

오늘 — GitHub 연동 벨트 ✓

- repo를 무대에 한 번 연결하면
- push할 때마다 알아서 새 버전이 오른다
- 현업 표준(CI/CD)을 그대로 쓰는 것

Vercel은 24시간 꺼지지 않는 극장 — 대본(코드)을 고칠 때마다 무대가 스스로 바뀐다.

배포하면 따라오는 3종 자동 선물

배포(deploy) = 내 앱을 남의 24시간 컴퓨터에 얹어 인터넷에 공개하는 것.

선물 ①

전 세계 유일 주소

내앱.vercel.app — 도메인이 자동 발급.

선물 ②

HTTPS 자물쇠

오가는 내용의 암호화. 7차 "설치·오프라인은 HTTPS에서만"의 그 자물쇠가 자동으로.

선물 ③

정적 사이트는 초간단

우리 앱은 조리(서버 계산)가 필요 없는 도시락 — 파일 그대로 내면 끝. 그래서 설정이 없다.

환경변수(비밀 값 금고) 같은 것도 있다 — 오늘은 필요 없는 이유를 [창고의 열쇠 이론](#)에서 설명.

자동배포, 그리고 되돌리기(롤백)

배포도 커밋처럼 역사가 남는다 — 각 배포 줄에는 커밋 메시지가 그대로 붙는다.

push

"커밋하고 push해줘" — 내가 하는 일의 전부

Vercel이 감지 → Building...

새 버전을 알아서 가져가 준비

Ready — 전 세계 공개

몇십 초 뒤, 같은 주소에 새 버전

망가졌다? → 클릭 한 번 롤백

직전 배포로 즉시 복귀

"무서워서 못 고친다"가 사라진다 — **기억(커밋)과 롤백**이 있으니까. 현업이 이것 없이 일 안 하는 이유.

처음 5클릭, 이후 0클릭

무대 연결은 처음 한 번뿐 — 이후엔 push가 전부다. (실행은 실습 Part 9에서)



무료(Hobby)는 개인·비사업 학습에 충분. 사업 서비스로 키울 땐 유료 전환 검토.

공유 링크 = DB의 존재 증명

오프닝의 그 장면 — 친구 폰에서 내 결과가 열리려면?

내 결과가 친구 폰에 보이려면,
그 데이터는 내 폰도 친구 폰도 아닌
제3의 장소에 있어야 한다.

localStorage의 감옥 — 폰 바꾸면 소멸 · 공유 불가 · 통계 불가. 논리적으로 불가능한 것이지, 기술이 부족한 게 아니다. 그 제3의 장소가 **DB(참고)**.

왜 파일이 아니라 DB인가

"그냥 파일에 저장하면 안 되나요?" — 세 가지가 무너진다.

① 동시성

100명이 동시에 저장

파일은 서로 덮어쓴다. DB는 줄 세워서 한 건도 안 잃는다.

② 검색

id 하나로 즉시

파일은 전부 읽어야 찾는다. DB는 바로 꺼낸다.

③ 규칙

DB가 지켜준다

"비면 안 됨", "4개 중 하나" — 규칙 위반은 저장 자체가 거절.

테이블 = 규칙 있는 엑셀 시트. 열 = 항목, 행 = 응답 1건. 만 명이 동시에 써도 안 깨지는 엑셀.

SQL — 참고에 말 거는 언어, 동사는 4개

속은 PostgreSQL — 대기업들이 쓰는 그 엔진을 무료로 빌린다. 오늘 SQL은 강사 스크립트 복붙 한 방.

INSERT · 넣기 ☒ 오늘 씀

SELECT · 꺼내기 ☒ 오늘 씀

UPDATE · 고치기 — 안 씀

DELETE · 지우기 — 안 씀

results 테이블 — 우리 참고의 설계 (2차 I/O 계약의 데이터판)

id	자동 발급 고유번호 → 이게 곧 공유 링크의 <code>?id=</code>
result	결과 유형 이름 (통계용)
payload	결과 전체 — 자유 구조 (질문이 몇 개든 OK)
created_at	저장 시각 (자동)

열쇠 2개와 문지기 RLS

참고가 인터넷에 있다 = 아무나 문을 두드릴 수 있다. 그래서 열쇠와 규칙이 필요하다.

anon 키 — 손님용 방문증 ✓

- 코드에 넣어 공개돼도 된다
- 뭘 할 수 있는지는 문지기(RLS)가 정한다
- 오늘 config.js에 넣는 게 이 열쇠

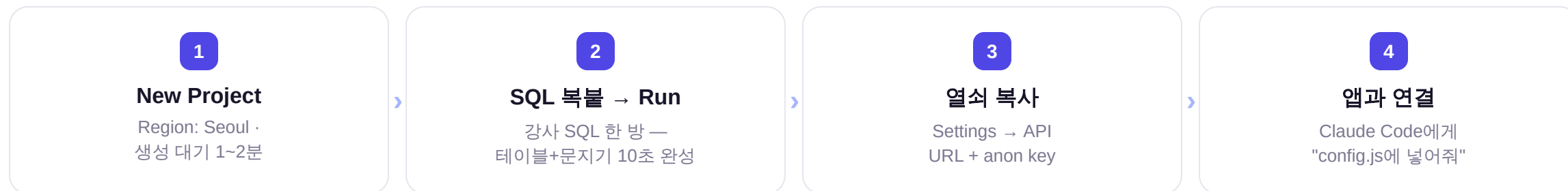
service_role — 마스터키 ✗

- 문지기를 무시하고 다 할 수 있다
- 절대 코드·GitHub에 올리지 않는다
- 오늘은 꺼내지도 않는다

문지기 규칙표(RLS): 넣기 ○ · 읽기 ○ · 고치기 ✗ · 지우기 ✗ → anon 키가 공개돼도 남의 결과를 조작할 수 없다. 습관: "공개돼도 되는 열쇠인지 모르겠으면, 공개하지 마라."

참고 개설 4단계

테이블을 만들면 "넣기/꺼내기 창구(API)"가 자동으로 생긴다 — 코드에선 한 줄로 쓴다. (실행은 Part 10)



⚠ 무료 프로젝트는 **약 1주일 미사용 시 일시정지** — "몇 주 뒤 안 열려요"의 1순위 원인. 대시보드에서 Restore 클릭이면 복구.

세 집이 한 몸이 되는 순간

오늘의 마스터 그림 — 이 한 장이 남으면 성공.



이 그림 어디에도
'켜져 있는 내 컴퓨터'는 없다.

데이터의 여행 — 친구가 링크를 여는 순간

`my-test.vercel.app/?id=abc123` — 이 한 번의 클릭에 세 집이 다 움직인다.

① 친구 폰 → 무대(Vercel)

주소로 요청 → 앱 화면이 내려온다

② 앱 → 참고(Supabase)

id 'abc123'으로 select 요청

③ 참고 → 친구 폰

내 결과 행을 내어줌 → 화면에 그려짐

문답 ①

코드를 망가뜨렸다?

기억(커밋 복귀) 또는 무대(롤백)

문답 ②

1만 명이 몰렸다?

일하는 건 무대와 참고 — 내 컴퓨터 아님

문답 ③

응답을 엑셀로?

참고 — Table Editor에서 내보내기

내 테스트를 기획한다 — Claude와 대화

강사 제공 '기획 프롬프트'를 claude.ai에 붙여넣고 대화 시작. 이 프롬프트, 어디서 본 틀이죠? — R-PCCO(1차).

1

프롬프트 붙여넣기

취향 질문 2개에 답

2

주제 고르기

후보 5개 중 선택·변형
(동물? 음식? 방 안의 물건?)

3

단계별 설계

성향 축 → 유형 → 질문
단계마다 내가 확인

4

"완성" → 기획안.md

출력물을 키트 폴더의
기획안.md로 저장

자유 기획 — 단 **최소 요건 2개**: ① 결과에 유형/등급 개념 (통계가 성립하려면) ② 결과 공유 기능 (오늘의 킬러 장면).

백엔드 키트 위에 **바이브 코딩**

키트 폴더에서 claude 실행 → 구현 요청은 한 줄이면 된다.

구현 요청 (복붙)

"기획안.md를 읽고, CLAUDE.md의 규칙을 지켜서 이 심리테스트 서비스를 구현해줘."

올타리

CLAUDE.md가 지킨다

어떤 기획이 와도 뒷단 계약은 유지 — 6차 "AI한테 한 장"의 실전 회수.

계약

db.js 함수 3개

saveResult · loadResult · getStats — 앞단은 자유, 참고 출입은 이 문으로만.

정상!

공유 버튼은 회색

"참고 연결 대기 중" — 아직 참고가 없어서. Part 10에서 커진다.

 생성이 꼬여 시간을 잡아먹으면 — [참고샘플\(동물테스트\)로 갈아타고](#) 뒷단 진도는 함께 간다.

GitHub — 기억의 집 입주

이론 ②의 3동작을 그대로. 로그인(처음 한 번) → 올리기 → 확인.

- "GitHub에 로그인해줘 (gh auth login)"

브라우저 인증만 직접 — Logged in 확인

- "이 폴더를 GitHub 새 저장소로 올려줘. 이름은 my-test"

repo 생성 + 첫 push

- 브라우저에서 repo 확인

파일 목록 · 기획안.md도 함께 박제 — 기획의 역사까지 기억에

- 한 가지 고치고 → "커밋하고 push해줘"

commits에 두 번째 줄 + 빨강/초록 비교 화면

"방금 여러분 코드가 지구 반대편 데이터센터에 박제됐습니다."

Vercel — 무대에 서다

이론 ③의 5클릭을 그대로. 코드 작업 없음 — 전부 vercel.com 대시보드에서.



"7차랑 뭐가 다른지 아직 안 보이죠? — **Part 11**에서 보여드립니다."

Supabase — 창고를 잇다

이론 ④의 4단계를 그대로 실행한다.

1

New Project

Region: Seoul
대기 중 개념 복습

2

SQL Editor → Run

supabase_setup.sql
복붙 한 방

3

열쇠 복사

Settings → API
URL + anon key

4

"config.js에 넣어줘"

Claude Code가 연결

로컬에서 테스트 1회 → **Table Editor**에 내 응답이 한 줄 꽂혀 있다 — "방금 여러분 데이터가 창고에 들어갔습니다." (와우 ①)
그리고 공유 버튼이 켜졌다.

맥박 확인 + 공유 이벤트

오늘의 와우 모먼트 3연타.

- "커밋하고 push해줘" → Vercel Deployments 관람
Building... → Ready. "저는 Vercel에 손도 안 댔습니다. push가 방아쇠." (와우 ②)
- 폰에서 내 테스트 → 결과 → 공유 링크 복사
옆 사람과 링크 교환
- 남의 폰에서 내 결과가 열린다 (와우 ③)
이론 ④의 논리가 눈앞에서 증명되는 순간
- 통계가 살아 움직인다
"당신 유형은 23%" — 창고가 쌓일수록 숫자가 변한다. 단독방에 URL 공유!

● 마무리

몸 · 기억 · 무대 · 창고 — 그리고 맥박

"코드는 GitHub에 살고, 실행은 Vercel에 살고, 데이터는 Supabase에 산다. 셋 다 내 컴퓨터가 아니다."

습관 1 고치면 커밋 의미 있는 변경마다 세이프 포인트 — 미래의 나를 위한 쪽지와 함께.

습관 2 마스터키 금지 service_role 키는 절대 코드·GitHub에 올리지 않는다. 모르겠으면 공개하지 마라.

습관 3 배포 후 폰 확인 무대에 올랐으면 반드시 진짜 관객석(폰)에서 확인.

 공지: 무료 창고는 1주 미사용 시 일시정지(Restore로 복구) · 심화 숙제: 7차 PWA 5요소를 얹어 설치형으로 — 단, "이 서비스는 왜 웹이 정답이었나"부터 답하고 시작.

공유 링크가 되는 순간, 앱이 아니라 서비스다

오늘 배운 건 심리테스트가 아니라 — 무엇을 만들든 세상에 내보내는 표준 배관.
다음: 이 배관 위에 리더보드? 방명록? 예약 시스템? 무엇이든.